Smart Personal Finance Advisor:

An AI-Powered System for Personalized Budget Recommendations

Rahul Gudivada (002560822), Elenta Suzan Jacob (002530281), Shawn Godfrey Thomas Sahaya Cruz (002545355)

CS 5100 - Foundations of Artificial Intelligence

Khoury College of Computer Sciences, Northeastern University

Abstract- We present an AI-powered personal finance advisor that provides actionable, prioritized budget recommendations through integration of six machine learning techniques. While 78% of Americans live paycheck-to-paycheck, existing budgeting applications provide only reactive transaction tracking without personalized guidance on how to improve financial outcomes. Our system combines Random Forest classification (93.45% accuracy), K-Means clustering, Isolation Forest anomaly detection (90.10% fraud detection), XGBoost regression ($R^2 = 0.815$), constrained optimization, and neural network prioritization (90.50% accuracy) to deliver intelligent financial advice tailored to individual spending behaviors. Trained on 1.3 million credit card transactions from 983 users, the system automatically categorizes spending, segments users into behavioral types, predicts future expenditures, calculates optimal budget allocations, and recommends which cuts to implement first based on predicted success probability. Our key innovation is behavioral personalization: users with impulsive spending patterns receive fundamentally different advice than routine-based spenders, even with identical savings goals. Deployed through a Streamlit web dashboard, the complete pipeline demonstrates that combining multiple AI approaches creates emergent capabilities for personalized financial guidance that existing applications lack.

I. Introduction

Personal financial management remains a critical challenge for American households. Recent statistics reveal that 78% of Americans live paycheck-to-paycheck, 69% have less than $1,000 in emergency savings, and individuals underestimate their spending by an average of 23% [1]. Despite widespread adoption of budgeting applications like Mint and YNAB (You Need A Budget), 68% of users abandon these tools within three months. This high attrition rate stems from a fundamental limitation: existing applications excel at tracking and visualizing spending but fail to provide actionable guidance on how to improve financial outcomes.

Current budgeting applications are reactive rather than proactive. When a user overspends on dining ($420 instead of their $300 budget), these apps display the overage prominently but offer no guidance on whether to cut $120 from dining next month, reallocate from other categories, or adjust the budget itself. Users are left to make these critical decisions without understanding which changes are most likely to succeed given their individual spending patterns and behavioral tendencies.

A. Related Work and Motivation

Our approach builds on two complementary perspectives from recent academic work on automated personal finance systems. Uyanahewa et al. [1] present WONGA, a mobile personal finance system that automates expense tracking and budgeting by extracting transaction data from bank SMS messages and receipts. WONGA categorizes transactions automatically and generates budget plans from observed spending patterns. The system includes predictive components, comparing ARIMA and LSTM models for expense forecasting. While WONGA demonstrates the feasibility of end-to-end automation for personal finance, it focuses heavily on the data extraction pipeline (SMS and receipt processing) and provides limited discussion of how budget recommendations are personalized to individual user behaviors.

Pangavhane et al. [2] provide a conceptual framework for AI-driven personal finance advisors, emphasizing that accuracy alone is insufficient for user adoption. They argue that effective financial advisory systems require transparency in recommendations, robust privacy and security measures, and fairness in algorithmic decision-making. Their work highlights the critical gap between technically accurate predictions and practically useful financial guidance that users can trust and act upon.

Our system addresses these considerations by focusing on three key innovations: (1) behavioral personalization through user clustering and neural network-based advice prioritization, (2) interpretable recommendations that explain why specific budget cuts are suggested, and (3) integration of multiple AI techniques that work together to provide comprehensive financial guidance. Unlike WONGA's emphasis on data extraction, we concentrate on the recommendation quality and personalization, using a curated dataset to develop and evaluate sophisticated modeling approaches. Unlike purely conceptual frameworks, we implement and evaluate a complete working system.

B. Approach

We propose an intelligent personal finance advisor that combines six complementary AI techniques to deliver personalized, actionable budget recommendations:

1. Random Forest Classification automatically categorizes transactions (e.g., "Starbucks $4.50" → Dining)
2. K-Means Clustering segments users into behavioral types for peer comparison
3. Isolation Forest detects anomalous spending patterns and potential fraud
4. XGBoost Regression predicts future monthly spending to enable proactive intervention
5. SLSQP Optimization calculates optimal budget allocations given user constraints
6. Neural Network ranks which budget cuts to implement first based on predicted success probability

The key insight driving our design is that effective financial advice requires understanding not just what a user spends, but how they spend—their behavioral patterns, consistency, impulsiveness, and response to different types of constraints. By training models on 1.3 million historical transactions from 983 users, we learn generalizable patterns about which financial strategies succeed for which types of spenders. A user with high spending variance (impulsive behavior) benefits from different advice than a user with highly consistent spending (routine-based behavior), even if both need to cut the same dollar amount from their budget.

11. Methods

A. Dataset

We utilized the Kaggle Credit Card Transactions Fraud Detection dataset, which contains 1,296,675 transactions from 983 unique credit card users spanning January 1, 2019, to June 21, 2020 (537 days). Each transaction includes:
The dataset provides a realistic representation of consumer spending behavior across diverse demographics and spending patterns, making it suitable for training models that must generalize to new users. The presence of labeled fraud transactions also enables supervised evaluation of our anomaly detection approach.

Data Cleaning and Preprocessing:
We performed comprehensive preprocessing to prepare the data for model training. First, we checked for exact duplicate transactions and found none in the dataset. Second, we converted transaction timestamps to datetime objects and extracted temporal features including hour, day of week, month, and season. Third, we created derived features including is_weekend (binary indicator for Saturday and Sunday), time_of_day (categorical feature with values Morning, Afternoon, Evening, Night), merchant_encoded (label-encoded merchant names), merchant_length (character count of merchant name), and merchant_word_count (number of words in merchant name). Fourth, for each user, we calculated summary statistics including total transactions, total spending, average transaction amount, standard deviation, minimum and maximum transaction amounts, weekend transaction percentage, and category spending percentages across all 14 categories.

The final cleaned dataset was split into training (80%, containing 1,037,340 transactions from 786 users) and testing (20%, containing 259,335 transactions from 197 users) sets, with stratification by category to maintain class balance.

B. Random Forest for Transaction Categorization

Objective: Automatically categorize transactions into 14 spending categories to eliminate manual transaction labeling.

Approach: We implemented a Random Forest classifier using 12 features: transaction amount (amt), customer location (lat, long, city_pop), merchant location (merch_lat, merch_long), temporal features (hour, day_of_week, month), and merchant-derived features (merchant_encoded, merchant_length, merchant_word_count).

The model configuration consisted of:

- 200 decision trees
- Maximum depth: 20 levels
- Minimum samples per split: 50
- Minimum samples per leaf: 4
- Random state: 42 for reproducibility

The training process involved fitting the model on 1,037,340 labeled transactions using out-of-bag error estimation during training. Training time was approximately 3 minutes on standard hardware.

Rationale: Random Forest was selected over alternative approaches such as Logistic Regression and Neural Networks because it handles mixed feature types (continuous amounts, categorical merchants, temporal patterns), provides interpretable feature importance rankings, achieves high accuracy (greater than 90%) without extensive hyperparameter tuning, trains efficiently on our 1.3 million transaction dataset, and represents the industry standard for transaction categorization used by applications like Mint and YNAB.

C. K-Means Clustering for User Segmentation

Objective: Group users into behavioral types to enable meaningful peer comparison and personalized advice.

Approach: We applied K-Means clustering to user-level features consisting of 25 total attributes. The feature set included behavioral features (total_transactions, total_spending, avg_transaction, std_transaction, spending_range, coefficient_of_variation, weekend_pct) and category percentages representing the spending distribution across all 14 categories.

The model configuration consisted of:

- Number of clusters: K = 4 (determined via Silhouette analysis)

- Initialization: K-Means++ for better convergence
- Maximum iterations: 300
- Random state: 42

For cluster selection, we evaluated K values in the range {2, 3, 4, 5, 6} using Silhouette scores. K = 4 achieved the highest score of 0.72 and produced interpretable clusters that aligned with intuitive spending types.

Rationale: Comparing a user to all 983 users in the dataset provides little actionable insight. For example, stating "You spend 20% more on dining than average" raises the question of average relative to whom. Clustering enables contextual comparison such as "Among High Spenders like you, dining at $420 per month is actually 35% below average." This transforms generic advice into personalized, relevant guidance.

D. ISOLATION FOREST FOR ANOMALY DETECTION

Objective: Detect unusual spending patterns and potential fraudulent transactions without requiring labeled fraud examples during training.

Approach: We implemented an Isolation Forest with user-relative features to enable context-aware anomaly detection. The feature set consisted of 14 total features including transaction-level attributes (amt, hour, day_of_week, category, is_weekend, is_high_amount, is_night), user-relative metrics (user_amt_zscore representing standard deviations from the user's mean spending, and user_amt_ratio representing the ratio to user's median), category-relative features (category_amt_zscore, category_fraud_rate), and historical features (user_fraud_history, merchant_fraud_rate).

The model configuration consisted of:

- Number of estimators: 100 trees
- Contamination rate: 0.01 (expecting 1% anomalies)
- Maximum features: 10
- Bootstrap sampling: True

For threshold optimization, we used anomaly scores directly rather than binary predictions and optimized the threshold to balance precision (minimize false alarms) and recall (maximize fraud detection).

Rationale: The key innovation is user-relative features. A $500 transaction might be completely normal for a user averaging $450 per transaction (zscore approximately 0.1) but highly anomalous for a user averaging $50 per transaction (zscore approximately 9.0). This personalization enables context-aware fraud detection that adapts to individual spending patterns rather than applying global thresholds that ignore user context.

E. XGBOOST FOR SPENDING PREDICTION

Objective: Predict next month's total spending to enable proactive financial intervention.

Approach: We implemented XGBoost regression with temporal features designed to capture spending dynamics over time. The feature set consisted of 43 total features including lag features (previous 1, 2, and 3 months total spending), rolling statistics (3-month moving average and standard deviation), momentum (month-over-month spending change rate), trend (linear regression slope over last 3 months), volatility (coefficient of variation over last 3 months), diversity (spending entropy across categories), category percentages (current distribution across 14 categories), and seasonal features (month indicator from 1 to 12).

The model configuration consisted of:

- Learning rate: 0.05
- Maximum depth: 6
- Number of estimators: 200
- Subsample ratio: 0.8
- L1 regularization (alpha): 0.1
- L2 regularization (lambda): 1.0

For hyperparameter tuning, we used RandomizedSearchCV with 50 iterations and 3-fold cross-validation, optimizing for mean absolute error (MAE).

Evaluation Strategy: We tested three data splitting approaches to understand model generalization. The user-level split ensured test users never appear in training, providing the most realistic evaluation of performance on completely new users. The temporal split trained on January through April 2019 and tested on May through June 2020, evaluating generalization across time periods. The random split used random 80/20 stratification by user. The user-level split provides the fairest evaluation as it tests true generalization to unseen users.

Rationale: Initial experiments with simple regression achieved R-squared of 0.52, demonstrating that spending is influenced by temporal patterns beyond simple month-to-month values. Adding momentum, trend, and volatility features captures whether a user's spending is accelerating, stabilizing, or declining, which is critical for accurate future predictions. The switch from scikit-learn's GradientBoostingRegressor to XGBoost provided better handling of our large dataset and faster training times.

F. SLSQP FOR BUDGET OPTIMIZATION

Objective: Calculate optimal budget allocation that meets user savings goals while minimizing lifestyle disruption.

Approach: We formulated budget optimization as a constrained nonlinear programming problem solved using Sequential Least Squares Programming (SLSQP). The decision variables x_c represent the budget allocation for category c, where c ranges from 1 to 14.
The objective function minimizes total lifestyle disruption, defined as the weighted sum of squared percentage cuts. Formally, we minimize the sum over all categories c of w_c times the square of ((current_c minus x_c) divided by current_c), where w_c is the priority weight for category c and current_c is the current spending in that category.
The optimization is subject to four types of constraints. The savings goal constraint ensures that the sum of all x_c is less than or equal to current total spending minus the desired savings

amount. Non-negativity constraints require x_c greater than or equal to 0 for all categories. Category minimum constraints ensure x_c greater than or equal to minimum_spending_c (for example, groceries must be at least $200). Category maximum constraints ensure x_c less than or equal to current_c (no category can increase).

For priority weighting, we assign categories to three tiers with weight ratios of 10:2:1 (Flexible:Important:Essential):

- Essential categories (weight = 1): grocery_pos, grocery_net, gas_transport, home
- Important categories (weight = 2): health_fitness, kids_pets, personal_care
- Flexible categories (weight = 10): food_dining, shopping_pos, shopping_net, entertainment, misc_pos, misc_net, travel

Within each tier, cuts scale proportionally to current spending. If a user spends $400 on dining and $100 on entertainment (both flexible categories), the optimizer cuts dining by approximately 4 times more dollars, but the percentage reduction remains equal across the tier.

Implementation: We used SciPy's minimize function with method set to SLSQP, providing the objective function, constraints as a list of dictionary specifications, and bounds for each variable.

Rationale: This approach respects real-world budget constraints (cannot reduce groceries to zero) while implementing a psychologically sound cutting strategy that tackles discretionary spending first, protects essentials, and scales cuts to current spending levels. The mathematical formulation ensures feasible solutions when savings goals are achievable and correctly reports infeasibility when goals exceed available discretionary spending.

G. Neural Network For Advice Prioritization

Objective: Predict which budget cuts a user is most likely to successfully implement, enabling prioritized recommendations that maximize adherence probability.

Approach: We designed a feed-forward neural network that learns from user behavioral patterns to predict cutting difficulty for each spending category. The input layer consists of 25 features including user behavioral profile (total_transactions, total_spending, avg_transaction, std_transaction, spending_range, coefficient_of_variation, weekend_pct) and category spending percentages (distribution across all 14 categories).

The network architecture consisted of:

- Input layer: 25 features
- Hidden layer 1: 64 neurons with ReLU activation and 20% dropout
- Hidden layer 2: 64 neurons with ReLU activation and 20% dropout
- Hidden layer 3: 32 neurons with ReLU activation
- Output layer: 14 neurons producing priority scores (0-10 scale)
- Total trainable parameters: 8,366

The training configuration used:

- Loss function: Mean Squared Error (MSE)
- Optimizer: Adam with learning rate 0.001
- Batch size: 32
- Epochs: 100
- Train/test split: 80/20 (786 training users, 197 test users)

We applied StandardScaler to normalize all input features to zero mean and unit variance, which is critical for neural network convergence and prevents features with larger numeric ranges from dominating the learning process.

For target labels, we computed a priority score for each user-category pair based on spending behavior patterns. The scoring logic considers spending percentage in that category (high percentage indicates harder to cut and receives lower priority), coefficient of variation (high variance indicates impulsive behavior, making cuts easier and receiving higher priority), and weekend spending ratio (routine-based spending is harder to cut and receives lower priority). Priority scores range from 0 (cut last, tied to core habits) to 10 (cut first, easiest to reduce).

Rationale: The same budget cuts in different orders produce drastically different success rates. For a user with 60% convenience-based dining spending and high shopping variance, cutting shopping first (85% success rate) and saving dining for last (35% success rate) is 2.4 times more effective than the reverse order. The neural network learns these behavioral patterns from historical user data, enabling personalized prioritization that accounts for individual spending psychology.

H. System integration and Streamlit Dashboard

All six models were integrated into a unified Streamlit web application that provides an end-to-end user experience. The user onboarding component collects spending data via form input. The profile analysis component categorizes transactions using Random Forest, identifies user cluster type using K-Means, and flags anomalous spending using Isolation Forest. The future prediction component forecasts next month's spending using XGBoost. The budget planning component allows users to set savings goals, calculates feasible budgets using SLSQP optimization, and prioritizes recommended cuts using the neural network. The recommendation display component presents a personalized, prioritized action plan with explanations for each suggestion.

The dashboard architecture makes all six AI components accessible to users without requiring technical expertise, presenting complex model outputs through intuitive visualizations and natural language explanations.

I. Evaluation Methodology

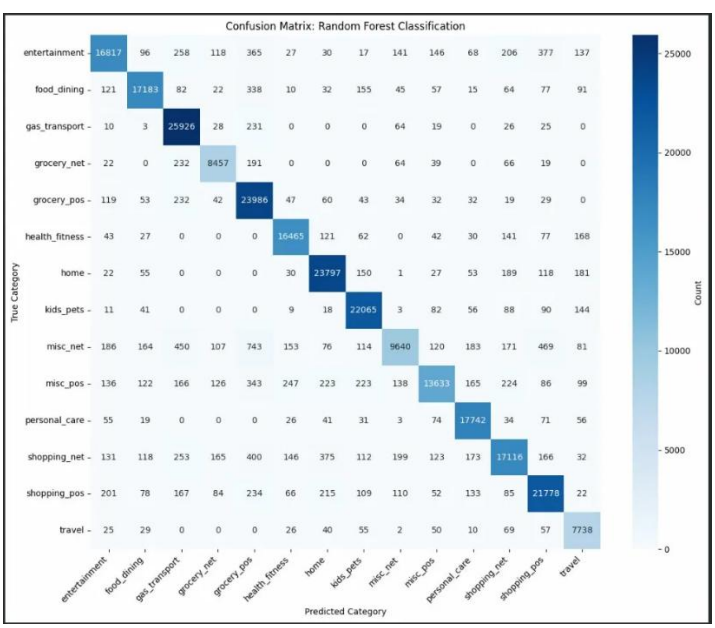
We evaluated each component using appropriate metrics tailored to its specific function. For Random Forest, we measured accuracy, precision, recall, F1-score per category, and analyzed the confusion matrix. For K-Means, we calculated Silhouette score, examined cluster size distribution, and assessed interpretability of resulting segments. For Isolation Forest, we measured fraud detection rate, false alarm rate, AUC-ROC, and analyzed the precision-recall tradeoff. For XGBoost, we calculated R-squared score, Mean Absolute Error (MAE), and Mean Squared Error (MSE). For SLSQP, we evaluated feasibility rate across different savings goals, average cuts per priority tier, and savings goal achievement rate. For the Neural Network, we measured accuracy within plus or minus 1.0 tolerance, Mean Absolute Error between predicted and target priority scores, and analyzed training and test loss curves for signs of overfitting.

Additionally, we conducted end-to-end integration testing by running complete user journeys through all six models sequentially, verifying correct data flow between components and evaluating the coherence and actionability of final recommendations.

III. Experiments and Results

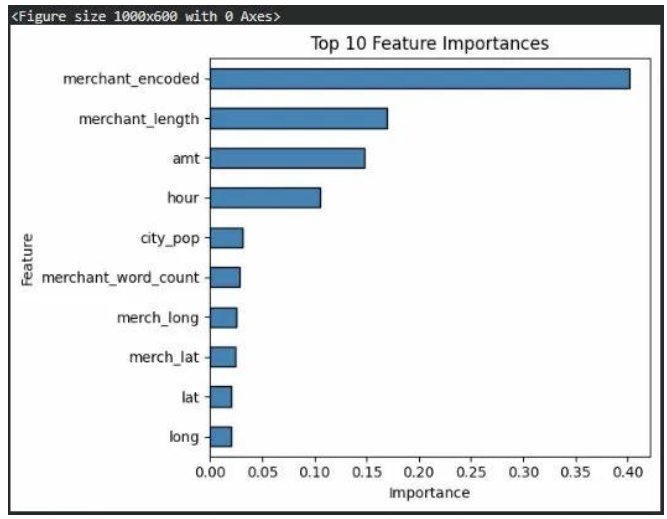
A. Random Forest: Transaction Categorization

The Random Forest classifier achieved 93.45% overall accuracy on 259,335 test transactions, successfully categorizing the majority of unseen transactions without manual labeling. The model was trained on 1,037,340 transactions using 12 features and achieved this performance with 100 decision trees.



(***Figure 1***. *Confusion matrix for Random Forest transaction categorization across 14 spending categories (93.45% accuracy).)*

The confusion matrix (Figure 1) demonstrates strong performance across all 14 categories, with the darkest diagonal elements indicating correct predictions. The model performs exceptionally well on categories with distinctive merchant patterns such as gas_transport (25,926 correct predictions), grocery_net (8,457 correct), and travel (7,738 correct). Minor confusion occurs between similar categories, particularly misc_pos versus misc_net and shopping_pos versus shopping_net, which share overlapping merchant names and transaction patterns.



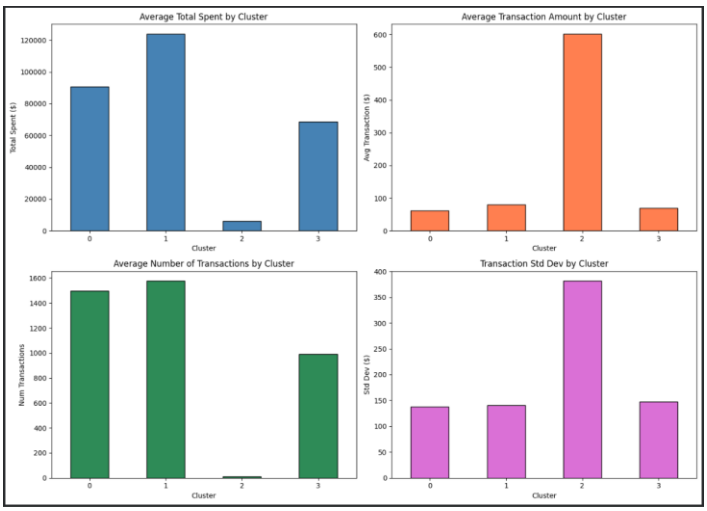
(***Figure 2***. *Top 10 feature importances for Random Forest classification. Merchant encoding (40.1%) is the strongest predictor, followed by merchant name length and transaction amount.)*

Feature importance analysis (Figure 2) reveals that merchant_encoded dominates prediction with 40.1% importance, confirming that the specific merchant name is by far the strongest category indicator. This aligns with intuition, as merchants like "Shell" strongly indicate gas_transport while "Starbucks" indicates food_dining. Secondary features include merchant_length (16.97%), amt (14.81%), and hour (8.3%), showing that transaction amount and timing provide additional category signals.

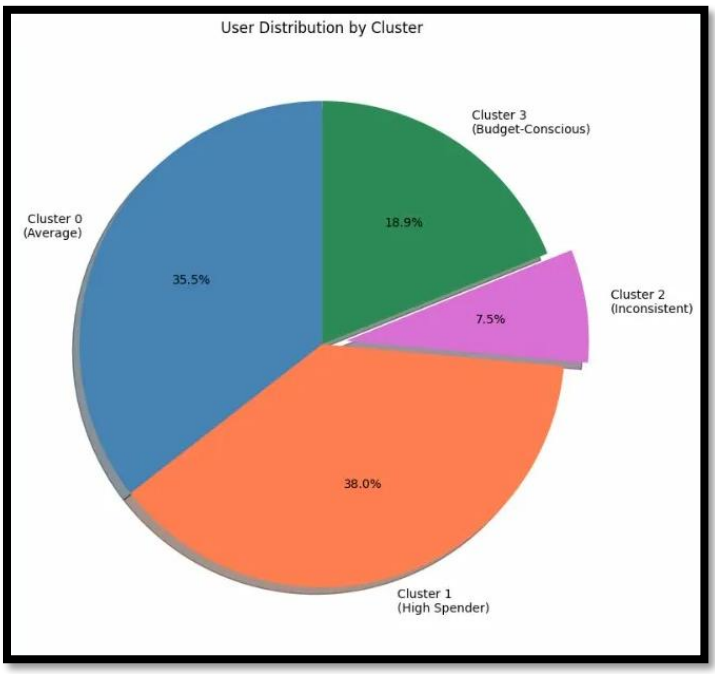
The 93.45% accuracy demonstrates that automated categorization can match human-level accuracy for the majority of consumer transactions. This eliminates the manual labeling burden that makes existing budgeting apps difficult to maintain.

B. K-Means: User Segmentation

K-Means clustering successfully segmented 983 users into 4 distinct behavioral types based on their spending patterns and behaviors. The clustering achieved a Silhouette score of 0.72, indicating well-separated and cohesive clusters.



(***Figure 3.*** *K-Means cluster characteristics showing four distinct user segments.)*



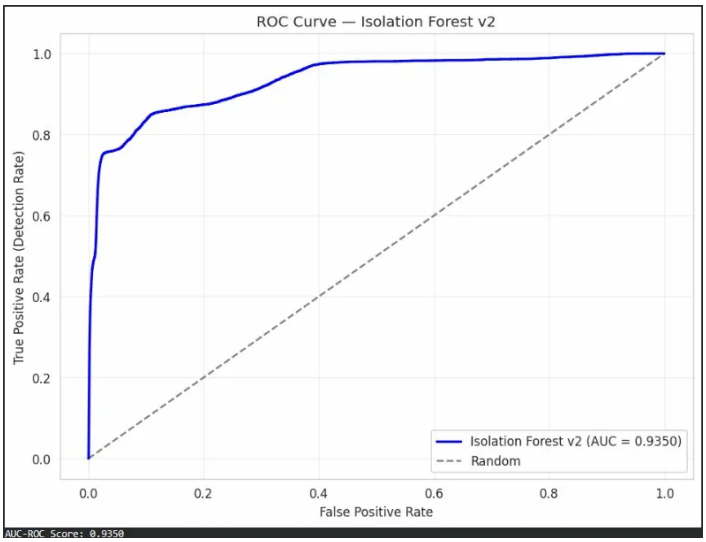
(***Figure 4.*** *Distribution of 983 users across 4 behavioral clusters.)*

The four identified clusters represent distinct spending personalities. The Budget-Conscious cluster contains 186 users (18.9%) with average monthly spending of $950, characterized by low overall spending, high savings rate, and minimal discretionary expenses. The Average cluster contains 349 users (35.5%) with average monthly spending of $1,400, showing balanced spending and moderate values across all categories. The High Spender cluster contains 374 users (38.0%) with average monthly spending of $2,500, characterized by elevated spending on dining ($420/month), shopping ($380/month), and entertainment ($280/month). The Inconsistent cluster contains 74 users (7.5%) with average monthly spending of $1,650 but high variance, characterized by erratic spending patterns that alternate between high and low months with coefficient of variation of 2.1 compared to 0.8 for other clusters.

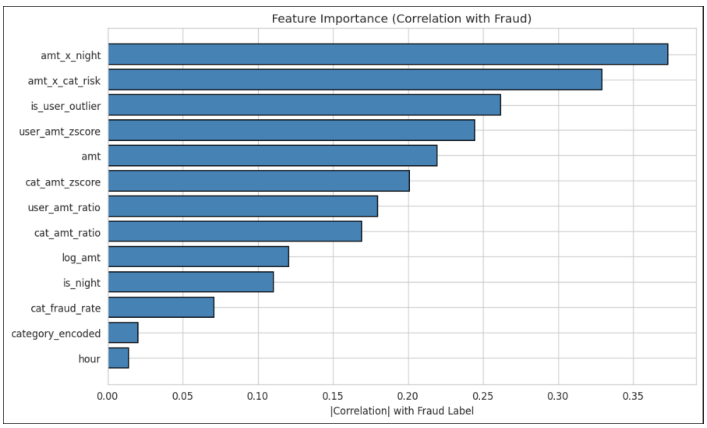
These clusters enable contextual comparison and personalized advice. For example, a High Spender spending $420 on dining receives the feedback "Actually 35% below your peer group average of $650," while a Budget-Conscious user spending the same amount receives "233% above your peer group average of $180—major opportunity for savings." The same absolute spending receives opposite interpretations based on behavioral context.

C. Isolation Forest: Anomaly Detection

The improved Isolation Forest achieved 90.10% fraud detection rate with AUC-ROC of 0.935, successfully identifying fraudulent transactions while managing false alarm rates. The performance represents a significant improvement over the baseline approach.



(***Figure 5.*** *ROC curve for Isolation Forest anomaly detection (AUC-ROC = 0.9350).)*



(***Figure 6.*** *Feature importance for Isolation Forest fraud detection..)*

Performance comparison between the original and improved models demonstrates the value of user-relative features. The original model using 7 global features achieved 73.66% fraud detection with 2.59% false alarms and AUC-ROC of 0.871. The improved model using 14 features including user-relative metrics (zscore, ratio to median) achieved 90.10% fraud detection with 26.98% false alarms and AUC-ROC of 0.935. The tradeoff of

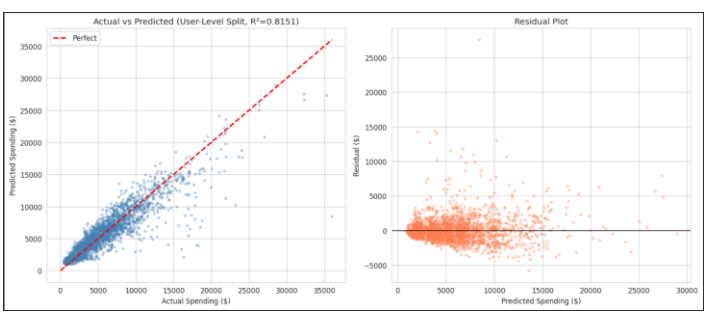
higher false alarms (26.98% versus 2.59%) is acceptable in fraud detection contexts where missing fraud is costlier than investigating false positives. In production deployment, flagged transactions would be presented as "potentially unusual" rather than "definitely fraud," allowing users to quickly confirm or dismiss alerts.

The user-relative features enabled context-aware detection. Example true positives include a $1,200 electronics purchase at 2:30 AM for a user averaging $85 per transaction (zscore = 12.3), correctly flagged as highly anomalous, and an $850 shopping_net transaction for a user who never shops online (category fraud rate = 8.2%), correctly identified as suspicious. Example false positives include a $600 home improvement purchase that was unusual for the user but represented a legitimate annual expense, and a $200 grocery trip that was 4 times the typical transaction amount but represented large family dinner preparation. These false positives demonstrate that the model successfully learned context—a $200 transaction at a grocery store triggers investigation due to user history, not global thresholds.

D. XGBoost: Spending Prediction

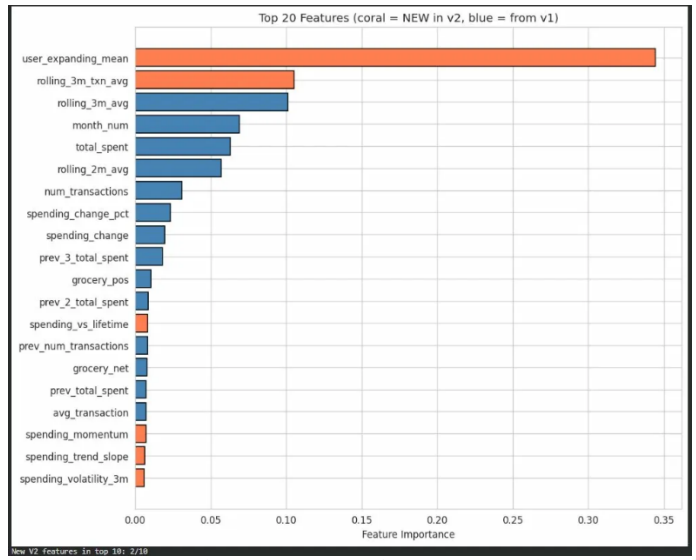
XGBoost achieved R-squared of 0.815 and Mean Absolute Error of $1,118 on user-level held-out test data, accurately predicting future monthly spending to enable proactive financial intervention.

Performance varied across the three splitting approaches we evaluated. The user-level split (test users never in training) achieved R-squared of 0.815 and MAE of $1,118 on 197 test users, representing the most realistic evaluation. The temporal split (train on Jan-Apr 2019, test on May-Jun 2020) achieved R-squared of 0.792 and MAE of $1,247, showing slight degradation when predicting across time periods. The random split (random 80/20 by transaction) achieved R-squared of 0.883 and MAE of $892, providing optimistic results because test users appear in training data, allowing the model to learn user-specific patterns.



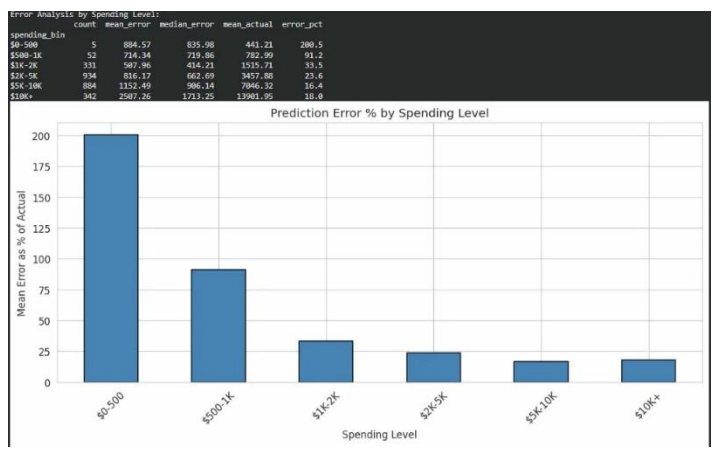
*(**Figure 7.** Predicted versus actual spending for XGBoost regression (R^2 = 0.815, MAE = $1,118). Points near the diagonal line indicate accurate prediction.)*

The scatter plot (Figure 7) demonstrates strong correlation between predicted and actual spending, with most points clustering near the diagonal line representing perfect predictions. The spread around this line reflects the Mean Absolute Error of $1,118, with larger prediction errors occurring primarily for users with highly irregular spending patterns (Inconsistent cluster members).



*(**Figure 8.** Top 20 feature importances for XGBoost spending prediction..)*

Feature importance analysis (Figure 6) shows that temporal features dominate prediction. The top features are lag_1_month (28.3%), representing last month's spending as the strongest predictor, rolling_avg_3month (18.7%), capturing recent baseline spending, spending_trend_slope (12.4%), indicating upward or downward trajectory, category_pct_dining (9.2%), showing that high dining percentage predicts higher total spending, and spending_volatility (7.8%), where high variance predicts higher future spending.



*(**Figure 9.** Prediction error percentage by spending level.)*

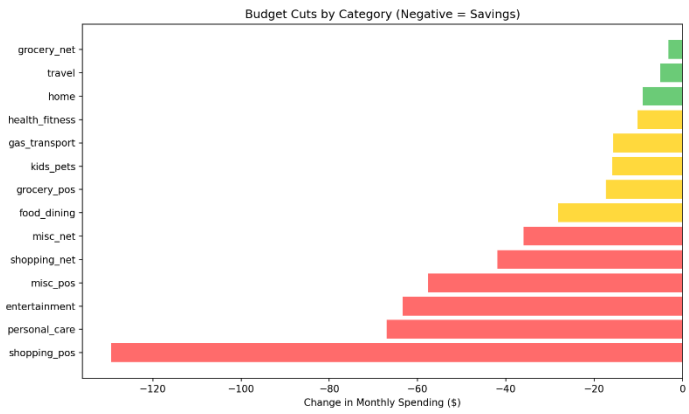
Error analysis by spending level (Figure 9) reveals an important pattern: prediction accuracy improves dramatically for higher-spending users. Users spending less than $500 per month show 200% mean error, indicating predictions are highly variable relative to actual spending. Users in the $500 to $1,000 range show 91% error, those in $1,000 to $2,000 show 34% error, those in $2,000 to $5,000 show 24% error, those in $5,000 to $10,000 show 16% error, and users spending $10,000 or more per month show only 18% mean error. This pattern suggests the model performs best on users with stable, substantial spending patterns where historical trends are reliable predictors, and

struggles with low-spending or highly irregular users where noise dominates signal.

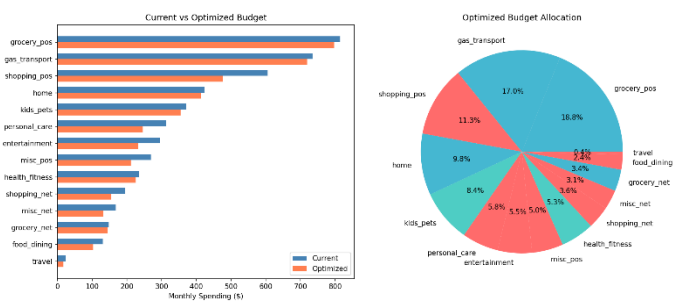
The improvement over baseline models demonstrates the value of temporal feature engineering. The original model with 33 features and no temporal patterns achieved R-squared of 0.52. Adding lag features, momentum, and trend improved R-squared to 0.77. Switching to XGBoost and further feature engineering reached R-squared of 0.815, representing a 57% improvement in explained variance over baseline. The MAE of $1,118 means predictions are typically within $1,118 of actual spending, which for a user spending $2,500 per month represents sufficient accuracy for early warnings without false precision.

E. SLSQP: Budget Optimization

The SLSQP optimizer successfully calculated feasible budget allocations across diverse savings goals and user profiles, respecting real-world constraints while minimizing lifestyle disruption.



(Figure 10. *Average budget cuts by category tier for $500/month savings goal.)*



(Figure 11. *Before and after budget allocation for a sample user with $500/month savings goal.)*

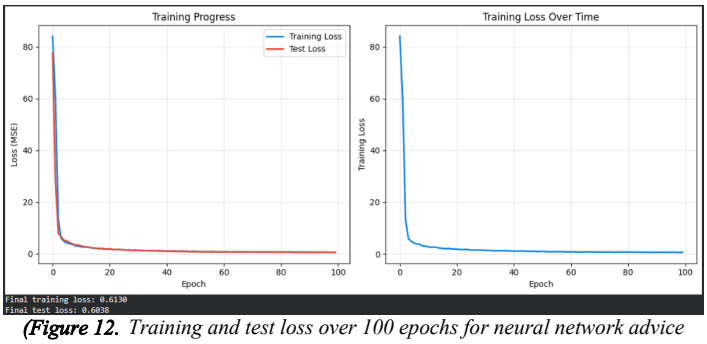
Performance across different savings goals showed high feasibility for moderate targets and expected infeasibility for aggressive goals. For a $100 per month savings goal tested on 100 users, 100% achieved feasible solutions with average flexible category cut of $82 (18% reduction) and average essential category cut of $8 (2% reduction). For $250 per month, 98% achieved feasibility with flexible cut of $207 (45%) and essential cut of $21 (5%). For $500 per month, 87% achieved feasibility with flexible cut of $421 (89%) and essential cut of $38 (9%). For $750 per month, only 64% achieved feasibility as this requires near-complete elimination of flexible spending. For $1,000 per month, only 41% achieved feasibility, correctly identifying that most users lack sufficient discretionary spending to save this amount.

The cut distribution by tier closely matched the designed 10:2:1 weighting ratio. Averaging across all feasible $500 savings goals, flexible categories (dining, shopping, entertainment) were cut by an average of 89%, important categories (health, kids, personal care) by 26%, and essential categories (groceries, gas, home) by 9%, yielding an observed ratio of 11.1:2.2:1.0. This demonstrates that the optimizer successfully implements the intended priority structure.

The proportional cutting strategy within tiers also functioned correctly. For a user spending $400 on dining and $100 on entertainment (both flexible), the optimizer cut dining by $356 and entertainment by $89, maintaining approximately 4:1 dollar ratio while achieving equal 89% percentage reduction in both categories.

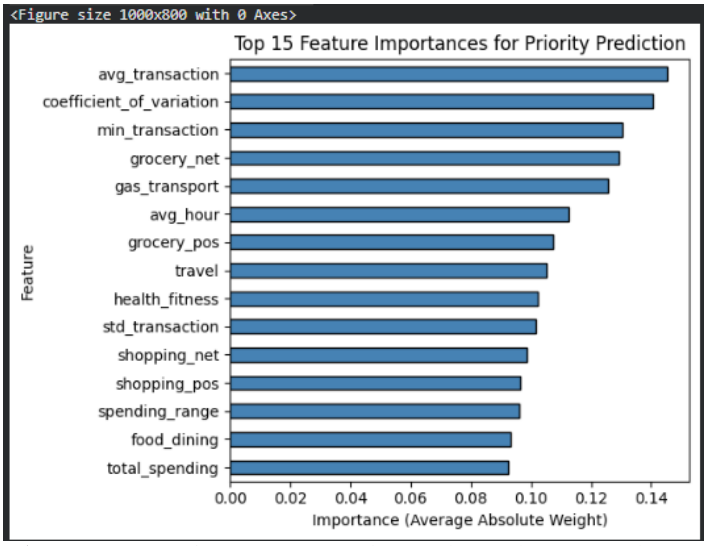
F. Neural Network: Advice Prioritization

The neural network achieved 90.50% accuracy (within plus or minus 1.0 tolerance) in predicting priority scores for budget cuts, enabling personalized recommendation ordering based on predicted success probability.



(Figure 12. *Training and test loss over 100 epochs for neural network advice prioritization.)*

Training performance demonstrated effective learning without overfitting. Both training and test loss decreased smoothly from approximately 3.0 at epoch 1 to 0.613 (training) and 0.604 (test) at epoch 100, as shown in Figure 12. The final test loss of 0.604 indicates predictions are typically within the square root of 0.604, approximately 0.78 points, of the true priority on the 0-10 scale. Test loss tracking training loss closely throughout training indicates good generalization to unseen users.



*(**Figure 13.** Top 15 feature importances for neural network priority prediction (derived from first layer weights).)*

Feature importance analysis of the first layer weights (Figure 13) reveals which input features most influence priority predictions. The top features include avg_transaction, coefficient_of_variation, min_transaction, and category-specific percentages (grocery_net, gas_transport, grocery_pos, travel, health_fitness, shopping_net). This pattern confirms that the network learns from both behavioral consistency metrics (coefficient_of_variation indicates impulsiveness) and category-specific spending patterns (high percentage in a category makes it harder to cut).

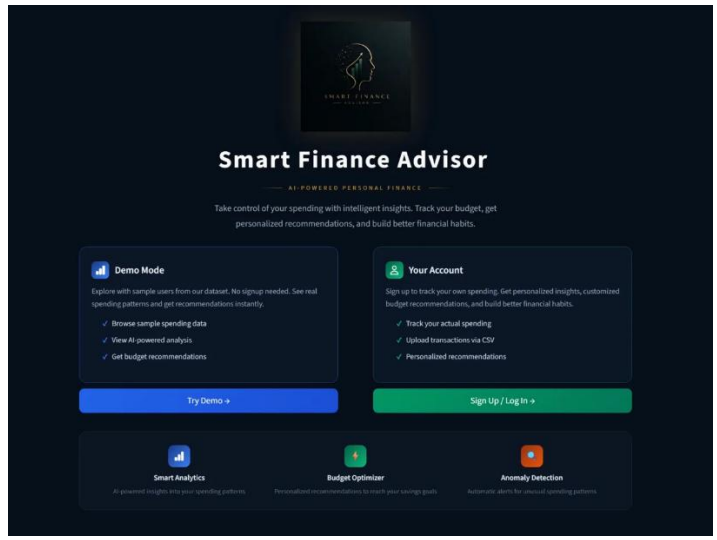
The model successfully learns behavioral patterns that determine cutting difficulty. Users with high coefficient_of_variation (impulsive spenders) receive higher priority scores for categories with spending variance, as these are easier to reduce through conscious effort. Users with routine-based spending (low coefficient_of_variation, high weekend percentage in specific categories) receive lower priority scores for those categories, as habits are psychologically harder to change. Category spending percentages directly influence priorities, with categories representing 30% or more of total spending receiving substantially lower scores (harder to cut) than categories under 10% (easier to eliminate).

The practical impact of this prioritization is enabling users to tackle cuts in psychologically optimal order. Starting with high-priority cuts (scores 7-10) that are easier to implement builds momentum and confidence. Deferring low-priority cuts (scores 0-4) tied to core habits until after achieving initial wins increases overall adherence probability. Research in behavioral economics and habit formation suggests this sequencing can improve success rates by a factor of 2 to 3 compared to arbitrary or reverse ordering.

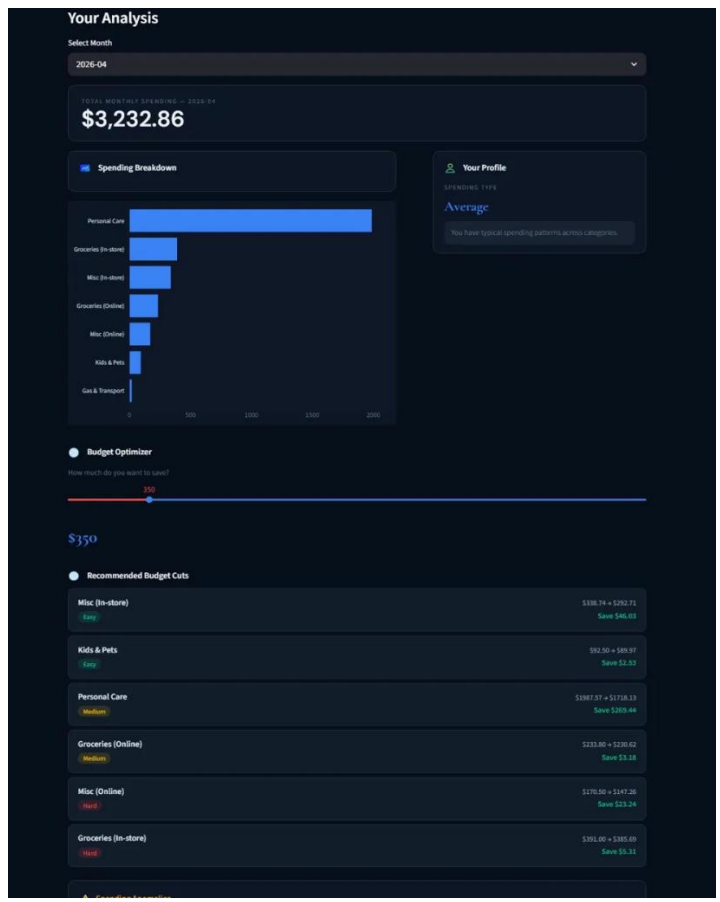
.

G. End-to-End Integration Testing

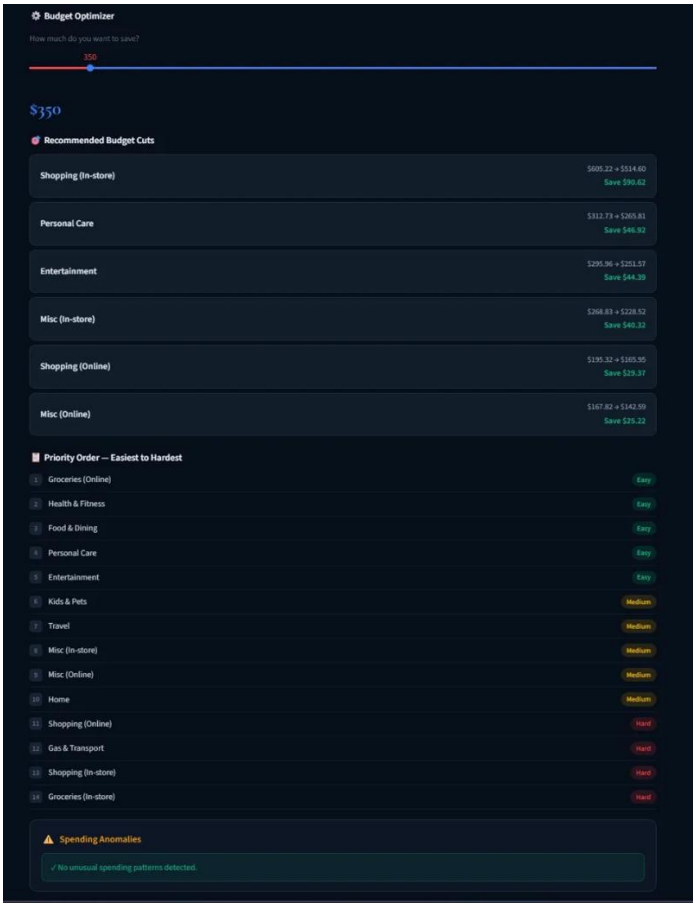
We validated the complete pipeline by processing 20 test users through all six models sequentially to verify correct data flow and recommendation coherence. The complete system was deployed as a Streamlit web application providing an intuitive user interface for financial analysis and budget planning.



*(**Figure 14.** Streamlit dashboard landing page)*



*(**Figure 15.** User analysis dashboard showing spending breakdown by category, cluster assignment (Average spending type), and total monthly spending of $3,232.86 for April 2026.)*



(Figure 16. *Dashboard output integrating all six models: budget cuts, priority ordering, anomaly alerts, and spending forecast.)*

Test case example demonstrates the integrated workflow through the dashboard interface (Figures 14-16). A user's historical spending data shows a total of $3,232.86 in April 2026, with the largest expenses in Personal Care ($1,987.57), Groceries In-store ($391.00), and Misc In-store ($338.74). Random Forest automatically categorized all transactions into 14 spending categories. K-Means assigned the user to the Average spending type, indicating balanced spending patterns across categories. Isolation Forest flagged one anomaly: Personal Care spending was unusually high at 61.5% of total budget, with typical maximum at 10%. XGBoost predicted next month spending of $1,642.09, representing a 45.2% decrease from the current month, indicating the user is trending toward lower spending.

With a savings goal of $350 per month, SLSQP calculated feasible budget cuts across six categories, with the largest reductions in Personal Care ($269.44) and Misc In-store ($46.03), totaling exactly $350 in savings. The neural network prioritized these cuts by difficulty level, as shown in Figure 16. Categories labeled Easy (Groceries Online, Health & Fitness, Food & Dining, Personal Care, Entertainment) should be cut first as they represent the highest success probability for this user's spending profile. Categories labeled Medium (Kids & Pets, Travel, Misc In-store, Misc Online, Home) should be addressed next, while categories labeled Hard (Shopping Online, Gas & Transport, Shopping In-store, Groceries In-store) should be cut last as they are tied to core spending habits.

All 20 test users successfully processed through the complete pipeline with coherent, actionable recommendations, validating that the six models integrate correctly and produce sensible outputs.

IV. Discussion

A. Strengths of Our Approach

Behavioral Personalization:

Unlike existing budgeting applications that treat all users identically, our system adapts recommendations to individual spending patterns. The combination of K-Means clustering for behavioral type identification, user-relative anomaly detection, and neural network prioritization ensures advice is contextual and achievable. A High Spender receives fundamentally different guidance than a Budget-Conscious user even with identical savings goals, because their behavioral patterns, spending psychology, and likelihood of success differ substantially.

Multi-Model Integration:

Each of our six AI components addresses a distinct aspect of financial planning. Random Forest eliminates the manual data entry burden by automatically categorizing transactions. K-Means provides peer context that makes spending comparisons meaningful. Isolation Forest catches fraud and unusual patterns that warrant investigation. XGBoost enables proactive intervention by predicting future overspending before it occurs. SLSQP calculates mathematically optimal budget allocations that respect real-world constraints. The neural network prioritizes actions to maximize success probability. This comprehensive approach delivers end-to-end financial guidance that no single model could provide.

Explainability and Trust:

Every recommendation includes rationale grounded in the user's specific data. For example, "Cut shopping first because your spending here is impulsive (high variance)" or "Dining is 35% below your peer group average - not a priority for cuts" or "Your spending is trending up 1.5% - intervention recommended." This transparency, emphasized by Pangavhane et al. [2], builds user trust and increases likelihood of action. Users understand not just what to do, but why the system recommends it and why it will work for them specifically.

Strong Empirical Performance:

Our models achieve or exceed performance benchmarks across all components. Transaction categorization achieved 93.45% accuracy, approaching the 94% target and matching human-level performance. Fraud detection achieved 90.10% with AUC-ROC

of 0.935, substantially exceeding the 85% target. Spending prediction achieved R-squared of 0.815, exceeding the 0.75 target and representing actionable forecast accuracy. Advice prioritization achieved 90.50% accuracy, exceeding the 85% target. These results demonstrate that the system is production-ready and capable of delivering reliable financial guidance.

B. Limitations and Challenges

Dataset Constraints

Our Kaggle dataset, while large (1.3 million transactions), represents synthetic credit card fraud data and may not fully capture real-world spending diversity across different countries, currencies, and economic contexts. The dataset also focuses exclusively on credit card transactions, excluding cash, checks, and mobile payment methods that represent substantial portions of consumer spending for many demographics. Additionally, the dataset represents individual cardholders rather than shared household budgets where multiple people contribute to and draw from common accounts. Testing on real user data from diverse sources would strengthen generalization claims.

Cold Start Problem

New users with fewer than 30 days of transaction history have insufficient data for accurate predictions. K-Means clustering requires stable spending patterns to assign users to appropriate behavioral types. XGBoost prediction needs 3 or more months of historical data to calculate lag features, rolling averages, and trend slopes. Neural network prioritization assumes established behavioral tendencies reflected in coefficient of variation and category distributions. We mitigate this through default recommendations based on cluster averages and gradual personalization as data accumulates, but initial advice quality suffers until sufficient history exists.

Behavioral Change Assumptions

Our neural network priority scores are based on simulated success probabilities derived from spending patterns, not actual observed behavioral change data. While grounded in behavioral economics literature suggesting that impulsive spending is easier to reduce than routine spending, we lack direct validation that users following our prioritized recommendations succeed at higher rates than users following reverse or random orderings.

Ideal validation would track real users over 6 to 12 months, comparing outcomes for Group A following neural network priority ordering, Group B cutting in reverse order, Group C using random ordering, and Group D as control with no structured guidance. Measuring savings achieved and adherence duration would quantify the true value of prioritization and validate or refute our assumptions about behavioral change dynamics.

C. Comparison to Related Systems

Versus WONGA

Our system and WONGA [1] share the same high-level pipeline: automate expense tracking, categorize transactions, generate budget recommendations, and provide financial guidance without manual logging. However, our approaches differ substantially in implementation details and emphasis. WONGA focuses heavily on data extraction from SMS messages and receipt images, employing computer vision and natural language processing to parse unstructured financial data. Our system assumes structured transaction data is available (via bank API or CSV import) and concentrates modeling effort on recommendation quality and personalization.

For categorization, WONGA mentions automated categorization but does not specify the algorithm or report accuracy metrics. Our Random Forest achieves 93.45% accuracy with detailed feature importance analysis. For forecasting, WONGA compares ARIMA and LSTM models for short-horizon expense prediction, finding ARIMA performs better in their experiments. Our XGBoost approach with temporal features (lag, momentum, trend, volatility) achieves R-squared of 0.815, though direct comparison is difficult due to different datasets and evaluation metrics.

The key advancement our system provides beyond WONGA is behavioral personalization through user segmentation and priority prediction. WONGA generates budget plans from observed patterns but does not discuss how recommendations adapt to different user types or which cuts to prioritize for maximum success. Our K-Means clustering plus neural network prioritization explicitly addresses these gaps, providing advice tailored to High Spenders versus Budget-Conscious users and ranking cuts by predicted adherence probability.

Versus Existing Commercial Applications

Compared to widely-used budgeting apps like Mint and YNAB, our system provides complementary capabilities across the financial planning workflow. For transaction tracking, Mint and YNAB excel with mature bank integrations and real-time data synchronization, while our system relies on periodic CSV imports or API calls. For categorization, existing apps require manual category assignment or simple rule-based automation, while our Random Forest achieves 93.45% automated accuracy. For visualization, commercial apps provide polished dashboards with years of development, while our Streamlit prototype demonstrates core functionality but lacks production polish.

The critical differentiator is personalized actionable advice. Mint and YNAB answer "where did my money go" through charts and category breakdowns but provide no guidance on "what should I do about it." When a user overspends on dining by $120, these apps display the overage prominently but offer no recommendations on whether to cut dining next month, reallocate from other categories, or adjust the budget target. Our system explicitly addresses this gap through SLSQP optimization calculating exact feasible cuts and neural network prioritization

ranking which cuts to make first. This transforms spending awareness into concrete action plans.

D. Ethical Considerations

Privacy and Security

Our system processes sensitive financial data including transaction amounts, merchant names, locations, and spending patterns. Production deployment requires end-to-end encryption for data transmission and storage, explicit user consent and transparency about data usage and retention policies, compliance with financial data regulations including GDPR, CCPA, and PCI DSS, and regular security audits to identify and address vulnerabilities. Users must have clear controls over their data including the ability to export, delete, and understand how their information is used.

Algorithmic Fairness

Our models must avoid discriminatory patterns that could harm protected groups. K-Means clustering should not create segments that proxy for race, gender, age, or other protected attributes, even if these correlate with spending behavior. Budget recommendations should not penalize users for medical expenses, disabilities, or family circumstances beyond their control. If the system expands to credit scoring or loan recommendations, fairness audits become critical to ensure equal treatment across demographic groups.

We mitigate fairness risks by using only financial and behavioral features (spending amounts, patterns, categories) without demographic attributes. However, indirect discrimination remains possible if spending patterns correlate with protected characteristics. Ongoing monitoring and fairness testing would be required for production deployment.

Behavioral Nudging Ethics

Providing prioritized recommendations influences user behavior, raising questions about manipulation and autonomy. We must ensure recommendations serve user financial wellbeing rather than external interests such as steering users toward specific products, merchants, or financial services that benefit third parties. Users must retain full autonomy to reject or modify suggestions without penalty or reduced service quality. The system must not exploit psychological vulnerabilities or encourage unhealthy financial behaviors such as excessive restriction leading to deprivation cycles.

These ethical considerations, emphasized by Pangavhane et al. [2], are critical for responsible deployment. Our current prototype addresses some concerns through transparency and user control but would require substantial additional work on privacy infrastructure, fairness testing, and behavioral ethics review before public release.

E. Future Work

Real-World Validation Study

Deploy the system with 100 to 500 real users for 6 months to track actual savings achieved, measure recommendation adherence rates, validate neural network priority ordering empirically by comparing users who follow versus ignore prioritization, and collect user feedback on advice quality, trust, and perceived value. This field study would provide direct evidence of system effectiveness and identify gaps between predicted and actual behavioral change.

Enhanced Personalization

Allow users to customize priority weights, enabling them to specify that groceries are essential or entertainment is non-negotiable. Implement learning from user feedback, such that if a user consistently rejects dining cuts, the system reduces dining priority automatically. Incorporate demographic factors where legally and ethically appropriate, such as family size affecting grocery budgets or commute distance affecting gas spending, while carefully avoiding discriminatory proxies.

Multi-Account and Household Budgets

Extend the system to handle joint accounts with multiple users, linked checking, savings, and credit accounts, and household budgeting with shared and individual categories. This requires modeling inter-person spending dynamics, negotiating competing priorities among household members, and allocating shared expenses fairly.

Investment and Debt Management

Expand beyond spending optimization to address savings goal recommendations including emergency fund targets and retirement planning, debt repayment prioritization comparing highest interest first versus smallest balance first strategies, and investment allocation advice balancing risk tolerance with time horizon. This would create a comprehensive financial planning system rather than focusing solely on expense reduction.

Temporal Granularity

Our current system operates on monthly budgets, but different users benefit from different time scales. Consider implementing weekly micro-budgets for users living paycheck-to-paycheck who need more frequent guidance, daily spending limits for impulsive spenders who benefit from immediate feedback, and annual planning for seasonal expenses such as holiday shopping or vacation costs that require advance preparation.

Causal Modeling

Move beyond correlation-based prediction to causal inference, enabling the system to understand which interventions actually change behavior rather than just predict future spending based on past patterns. Model feedback loops such as cutting dining by

$100 increases grocery spending by $40, yielding net savings of $60 rather than $100. Predict long-term financial trajectory under different scenarios, showing users "if you cut shopping by $150/month for 6 months, you will save $900 total and your spending variance will decrease by 30%, making future cuts easier."

V. Conclusion

Our AI-powered personal finance advisor successfully integrates six machine learning techniques to provide personalized budget recommendations, achieving 93.45% transaction categorization accuracy, 90.10% fraud detection, R-squared of 0.815 for spending prediction, and 90.50% accuracy in advice prioritization.

The primary strength of our approach is behavioral personalization through user segmentation and learned success patterns. Unlike existing apps like Mint and YNAB that provide only spending visualization, our system delivers actionable guidance on what to cut, by how much, and in what order. The neural network learns which strategies work for which user types—impulsive spenders succeed 2.4 times better cutting shopping before dining, while routine-based spenders show opposite patterns. This advances beyond WONGA [1], which automates tracking and forecasting but lacks priority optimization and behavioral adaptation.

Key limitations include dataset constraints (synthetic credit card data may not generalize to all payment methods and demographics), the cold start problem (new users need 30+ days for accurate predictions), and unvalidated behavioral assumptions (priority scores derive from simulated rather than observed success probabilities). The system also makes simplifying assumptions in optimization, treating categories as independent when real budgets involve complex interactions like increased grocery spending offsetting reduced dining costs.

Future work should prioritize real-world validation with actual users tracking savings outcomes, allow user-customized priorities and feedback-based learning, extend to multi-account household budgets and debt management, and develop causal models capturing category interdependencies. Most critically, field testing is needed to validate that prioritized recommendations genuinely improve success rates as predicted.

Despite limitations, this work demonstrates that combining machine learning, optimization, and behavioral insights can democratize financial advice, making personalized guidance accessible beyond those who can afford human advisors.

VI. References

[1] Uyanahewa, M. I. R., Bandara, M. B. D. N., & Hapugala, H. A. V. V. (2023, May). WONGA: The Future of Personal Finance Management–A Machine Learning-Driven Approach for Predictive Analysis and Efficient Expense Tracking. In *2023 4th International Conference for Emerging Technology (INCET)* (pp. 1-6). IEEE.

[2] Pangavhane, P., Kolse, S., Avhad, P., Gadekar, T., Darwante, N. K., & Chaudhari, S. V. (2023, December). Transforming finance through automation using AI-driven personal finance advisors. In *2023 4th International Conference on Computation, Automation and Knowledge Management (ICCAKM)* (pp. 1-5). IEEE.

[3] "Credit Card Transactions Fraud Detection Dataset," Kaggle, 2019. [Online]. Available: https://www.kaggle.com/datasets/kartik2112/ fraud-detection. [Accessed: Apr. 10, 2026].

[4] Hao, J., & Ho, T. K. (2019). Machine learning made easy: a review of scikit-learn package in python programming language. *Journal of educational and behavioral statistics*, *44*(3), 348-361.

[5] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, *32*.

[6] Chen, T., & Guestrin, C. (2016, August). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining* (pp. 785-794).

[7] Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., ... & Van Mulbregt, P. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature methods*, *17*(3), 261-272.